



2E/9.1.3

## III (Even) Semester Regular/Back Examination May 2026

Roll no. 2394076

Name of the Program and semester: B Tech CSE, VI Sem

Name of the Course: Compiler Design

Course Code: TCS 601

Time: 3 hours

Maximum Marks: 100

### Note:

- (i) All the questions are compulsory.
- (ii) Answer any two sub questions from a, b and c in each main question.
- (iii) Total marks for each question is 20 (twenty).
- (iv) Each sub-question carries 10 marks.

Q1.

(2X10=20 Marks) CO1

a. What are the phases of compilers? Categorize them into front end and back end of the compiler, Discuss the action taken by every phase of the compiler on the following string:

$A + B - D * E / F$

b. Explain the need for input buffering in lexical analysis. Consider a source program stored in a file Design a simulation (in pseudocode) of the input buffer handling using the two-buffer scheme. Clearly explain how buffer switching and token formation are handled.

c. Design a Lex specification to recognize and tokenize the following tokens in a programming language:

Keywords: if, else, while, for

Identifiers: alphanumeric strings starting with a letter

Integers: decimal numbers

Operators: +, -, \*, /, =, ==, !=

Symbols: (, ), {, }, [, ]

Write the Lex code to recognize these tokens and print the token type and value.

Q2.

(2X10=20 Marks) CO2

a. Explain with example how precedence and associativity help in resolving ambiguity in arithmetic expressions.

b. Construct LL (1) Parsing table for given grammar:

$S \rightarrow T * P$

$T \rightarrow U / T * U$

$P \rightarrow Q + P / Q$

$Q \rightarrow id$

$U \rightarrow id$

c. Define canonical collections of LR(0) items and LR(1) items. Consider the given grammar productions:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Verify whether the given grammar is LR(0) and LR(1) or not by constructing canonical collection of LR(0) items.



**End Term (Even) Semester Regular/Back Examination May 2026**

(2X10=20 Marks) CO3

Q3.

- a. What do you mean by syntax-directed definition? Explain synthesized and inherited attributes in detail.
- b. Consider the following Syntax-Directed Translation schemes.

Grammar production

$L \rightarrow S$

$S \rightarrow aSb$

$S \rightarrow bSa$

$S \rightarrow ab$

Semantic rules

{ printf( "A" ); }

{ printf( "B" ); }

{ printf( "C" ); }

{ printf( "D" ); }

If we carried out the above SDT on the given input string "ababab" then what will be the final output printed by it? Illustrate your explanation by using both top-down and bottom-up approach.

- c. Explain the application of SDT to convert infix expression to prefix expression.

(2X10=20 Marks) CO4

Q4.

- a. Consider the following three-address code:

1.  $a = b + c$

2.  $d = a - e$

3. if  $d > 0$  goto 7

4.  $e = d * 2$

5. goto 8

6.  $a = b - c$

7.  $d = a + e$

8. print d

- (i) Identify leaders  
(ii) Form basic blocks  
(iii) Draw the flow graph

- b. Consider the following switch statements:

(i)  
switch(i + j)  
{  
  case 1:  $a = b + c$   
  default:  $p = q + r$   
  case 2:  $x = v + w$   
}

(ii)  
switch(ch)  
{  
  case 1:  $c = a + b$ ;  
    break;  
  case 2:  $c = a - b$ ;  
    break;  
}

Write the **three-address code** for the above switch cases.

- c. Explain the importance of **evaluation order of expressions** in minimizing register usage.



# Graphic Era

HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)  
University under section 2(f) of UGC Act 1956

GEHU/02E/9.1.3

## End Term (Even) Semester Regular/Back Examination May 2026

(2X10=20 Marks) COS

Q5.

a. How are error handling and recovery mechanisms implemented in different phases of a compiler?

b. What do you mean by peephole optimization? What are its characteristics? Optimize the following code:

```
p = 0
```

```
i = 1
```

```
do
```

```
{
```

```
    p = p + A[i] * B[i]
```

```
    i = i + 1
```

```
}
```

```
while (i <= 20)
```

c. Explain how loop detection helps in code optimization techniques such as:

i. Loop invariant code motion

ii. Strength reduction

iii. Loop unrolling